

What is a System Call?

A system call is a mechanism used by programs to request services from the operating system (OS). It allows a program to interact with the system for tasks like accessing hardware or performing privileged operations.

A user program can interact with the operating system using a system call. A number of services are requested by the program, and the OS uses system calls to fulfill them.

A system call can be written in high-level languages like C or Pascal or in assembly language. If a high-level language is used, the operating system may directly invoke system calls, which are predefined functions.

When a program executes a system call, it transitions from user mode to kernel mode, allowing the operating system to perform the requested operation on behalf of the program.

System calls are crucial for an operating system as they let programs access system resources in a standardized way. Without them, programs would have to create their own methods to use hardware and services, causing inconsistencies and errors.

Services Provided by System Calls

- Process Creation and Management
- Main Memory Management
- File Access, Directory, and File System Management
- Device Handling(I/O)
- Protection
- Networking, etc.

Here is a step-by-step explanation of how system calls work:

1. **Users need special resources:** Sometimes, programs require operations that need the OS's permission, such as reading or writing files, accessing hardware, or requesting memory space.
2. **The program makes a system call request:** Programs use specific, predefined instructions known as "system calls" within their code to request these operations from the OS.

3. **Operating system sees the system call:** When the OS detects a system call, it pauses the program and passes control to a special component called the 'Kernel' to address the program's request.
4. **The operating system performs the operations:** The OS then executes the requested operation, like reading data from a file.
5. **Operating system gives control back to the program:** Once the operation is complete, the OS returns control to the program to continue its execution.

System Call Parameter Passing:

In system calls, parameter passing is how arguments are transferred from a program to the operating system when making a system call. Since the OS needs to understand what the program is asking for, the parameters provide specific details for the request, such as file paths, memory addresses, or permissions.

Common Methods of System Call Parameter Passing:

1. **Registers:** The simplest method where parameters are stored in CPU registers before making a system call. However, the number of registers is limited, so this method works best for calls with a small number of parameters.
2. **Memory Block:** If there are more parameters than registers can handle, parameters are stored in a memory block, and the address of this block is passed to the OS through a register. The OS can then access all the parameters by reading from this memory block.
3. **Stack:** Parameters are pushed onto the stack by the program making the system call. The OS can then retrieve these parameters directly from the stack when handling the call.

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS		
	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Advantages of System Calls:

- **Safe Resource Access:** Provides controlled access to hardware and OS resources.
- **Simplified Programming:** Abstracts complex operations, like file management.
- **Security:** Keeps critical operations secure within the OS.

Disadvantages of System Calls:

- **Performance Overhead:** Switching to the OS kernel can slow down performance.
- **Limited Portability:** OS-specific calls reduce application flexibility.
- **Complexity:** Frequent system calls can complicate program design.

What is Process? Give the difference between Process and Program.

A process is a program in execution. It includes the program's current activity, like the instructions being executed, the data being used, and the state of the CPU.

A **process** is an active entity, meaning it is a running instance of a program. It is different from a program, which is a passive set of instructions stored on disk.

Feature	Process	Program
Definition	An instance of a program in execution.	A set of instructions that can be executed by a computer.
Existence	Dynamic entity that exists for a limited time.	Static entity that exists until it is deleted.
Creation	Created when a program is executed.	Created when a program is written or compiled.
Resources	Requires resources such as CPU time, memory, and I/O devices.	Does not require any resources.
State	Can be in one of several states, such as new, runnable, running, blocked, or terminated.	Always in the same state.
Control block	Has a process control block (PCB) that contains information about the process.	Does not have a control block.
Execution	Can be executed concurrently with other processes.	Can only be executed one at a time.
Example	A web browser, a word processor, a game.	A source code file, an executable file.

Components of a Process in OS

The components of a process are:

Program code/Text: The instructions that the process will execute.

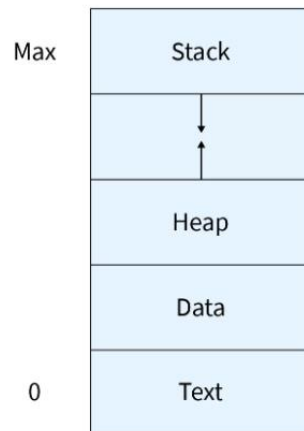
Data: The data that the process will use during its execution.

Stack: A data structure that is used to store temporary data, such as function parameters and return addresses.

Heap: A data structure that is used to store dynamically allocated memory.

Process control block (PCB): A data structure that contains information about the process, such as its state, priority, and memory usage.

Process In Memory:

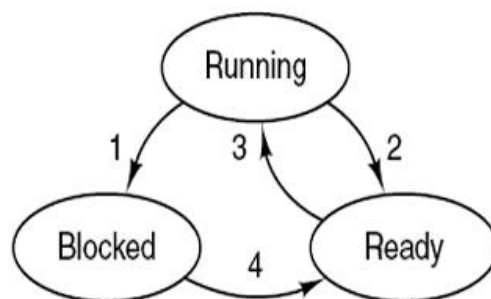


Process States:

As a process executes, it changes state. The state of a process is defined in part by the current activity of that process. A process may be in one of the following states:

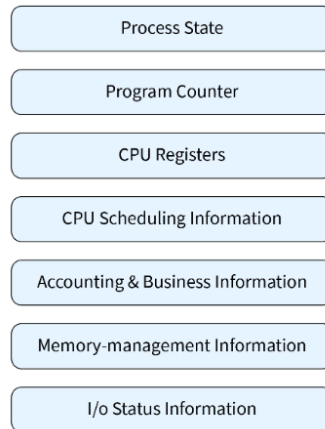
- **New:** The process is being created.
- **Running:** Instructions are being executed.
- **Waiting:** The process is waiting for some event to occur (such as an I/O completion or reception of a signal).
- **Ready:** The process is waiting to be assigned to a processor.
- **Terminated:** The process has finished execution

Process State Transition Diagram:



Process Control Block (PCB):

An OS serves in the creation, scheduling and termination of processes. The OS consists of Process Control Block (PCB) that helps control the functioning of processes. Every process in OS has a PCB associated with it. A PCB keeps track of processes by storing information about various things like their state, I/O status and CPU Scheduling.



A PCB comprises of the following:

1. **Process ID:** An identifier that helps us in identifying and locating a process.
2. **Process state:** It identifies the state that the process is currently in. It could be a new process, ready, running, waiting or terminated.
3. **Program counter:** It holds the address of the next instruction to be executed for the process.
4. **CPU registers:** The content of processor registers gets stored here when the process is in running state. The different kinds of CPU registers are accumulators, index and general-purpose registers, instruction register and condition code register.
5. **CPU scheduling information:** A process needs to be scheduled for execution. Based on this scheduling, it goes from ready to running.
6. **Accounting and business information:** Contains details like CPU utilization, real-time used by the process, number of jobs or processes, etc.
7. **Memory-management information:** Contains value of base and limit registers, details about page and segment tables. It depends on the memory system of the OS in use.
8. **I/O status information:** Comprises I/O related information including list of I/O devices allocated to the process, status, etc.

Threads:

A thread is the smallest unit of execution within a process. They represent a lightweight process, allowing multiple threads to exist within a single process, sharing the same resources but executing independently.

Components of Threads

These are the basic components of the Operating System.

- Stack Space
- Register Set
- Program Counter

Thread States:

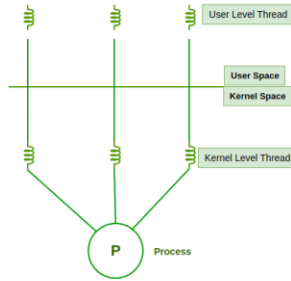
Similar to processes, threads can also be in various states:

- **New:** The thread is being created.
- **Runnable:** The thread is ready to run and waiting for CPU time.
- **Blocked:** The thread is waiting for a resource (like I/O or a lock).
- **Terminated:** The thread has completed its execution.

Types of Thread in Operating System

Threads are of two types. These are described below.

- **User Level Thread:** are threads managed entirely by user-level libraries without kernel involvement, allowing easy implementation but requiring kernel-level threads for single-handed processes.
- **Kernel Level Thread:** are recognized by the operating system and managed by the kernel using a special table. They take longer to switch between compared to user-level threads.



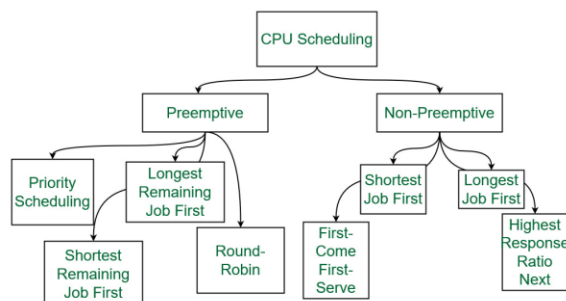
Benefits of using threads:

1. **Resource Efficiency:** Less overhead than creating new processes
2. **Better Performance:** Faster context switching between threads
3. **Enhanced Responsiveness:** Can handle multiple tasks simultaneously
4. **Resource Sharing:** Easy communication between threads

Uni-Processor Scheduling:

Uni-Processor Scheduling in an operating system refers to the method by which the OS manages the execution of processes or threads on a single CPU core. Since there's only one processor available, the OS must decide which process to run at any given time, effectively managing time and resources.

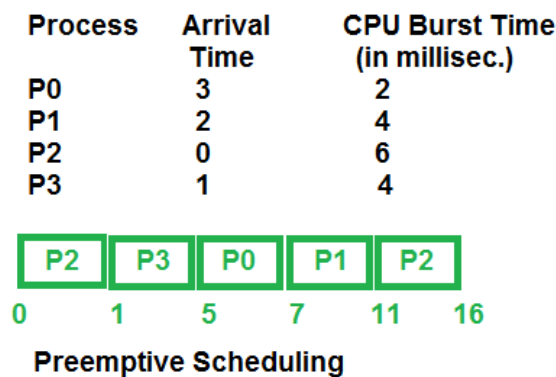
In operating systems, scheduling is the method by which processes are given access to system resources, primarily the CPU. Efficient scheduling is essential for optimal system performance and user satisfaction. There are two primary types of CPU scheduling: preemptive and non-preemptive.



What is Preemptive Scheduling?

Preemptive scheduling is used when a process switches from the running state to the ready state or from the waiting state to the ready state. The resources (mainly CPU cycles) are allocated to the process for a limited amount of time and then taken away, and the process is again placed back in the ready queue if that process still has CPU burst time remaining. That process stays in the ready queue till it gets its next chance to execute.

Algorithms based on preemptive scheduling are Round Robin (RR), Shortest Remaining Time First (SRTF), Priority (preemptive version), etc.



In preemptive scheduling, the CPU can be interrupted, unlike in non-preemptive scheduling.

Advantages of Preemptive Scheduling:

- Improves average response time.
- More suitable for multi-programming environments.
- Ensures fair CPU time distribution among processes.

Disadvantages of Preemptive Scheduling:

- Requires limited computational resources.
- Involves time overhead for suspending processes and context switching.
- Low-priority processes may be delayed if many high-priority processes arrive simultaneously.

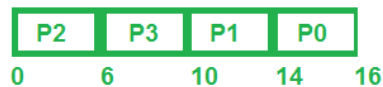
What is Non-Preemptive Scheduling?

Non-preemptive Scheduling is used when a process terminates, or a process switches from running to the waiting state. In this scheduling, once the resources (CPU cycles) are allocated to a process, the process holds the CPU till it gets terminated or reaches a waiting state.

In non-preemptive scheduling, a running process is not interrupted and continues until it completes its CPU burst time before the CPU is allocated to another process.

Algorithms based on non-preemptive scheduling are: Shortest Job First (SJF basically non preemptive) and Priority (nonpreemptive version), etc.

Process	Arrival Time	CPU Burst Time (in millisec.)
P0	3	2
P1	2	4
P2	0	6
P3	1	4



Non-Preemptive Scheduling